

Contents

Reward Design for LLM-Driven Truss Optimization	2
Quick orientation	2
0. Notation and abbreviations	3
0.1 Abbreviations	3
0.2 Mathematical notation	3
0.3 State feature vector	4
0.4 Action grammar and locality partition	4
1. Designing r_{env} from the engineering objective	5
1.1 The objective and its Lagrangian potential	5
1.2 The per-step reward	5
1.3 Properties	6
1.4 Margin preservation as an objective choice	6
1.5 What r_{env} does not credit	6
2. Modification trees: SFT priors and runtime pattern signals	7
2.1 State-regime action prior (SFT auxiliary)	7
2.2 Sibling-contrast preferences (SFT via DPO)	7
2.3 Macro-action mining (GRPO-time runtime pattern)	8
2.4 Dead-end avoidance (GRPO-time runtime pattern)	8
3. Online tree-expanded value estimation	9
3.1 The estimator	9
3.2 Theorem 1 — Bias bound for tree-expansion value	10
3.3 Theorem 2 — Ranking accuracy and branching bound	11
3.4 Branching coverage bound	12
3.5 Cost-benefit and practical default	13
3.6 Connection to LLM-advantage	14
3.7 NHR invariance check	14
4. LLM-capability shaping	14
4.1 Forward-prediction grounding	15
4.2 Difficulty-weighted return	15
4.3 Stagnation-escape bonus	15
4.4 Cross-problem strategy adaptation	16
5. Information attribution	16
6. The full reward function	16
6.1 Combined formula	16
6.2 Default coefficients	17
6.3 Recommended ablation sequence	18
7. Verification and benchmark	19
7.1 Per-component verification	19
7.2 Headline benchmark	19
7.3 Theoretical-bound verification	20
8. Open problems	20
8.1 Continuation policy choice in tree expansion	20
8.2 Tree-expansion overhead on training time	20
8.3 Inference-time gap	21
8.4 Reward-hacking surface	21
8.5 Cross-domain action grammar	21

8.6 Per-problem threshold generalisation	21
8.7 Branch-entropy as a lost signal	21
8.8 SFT prior staleness	22
9. Recommended reading	22

Reward Design for LLM-Driven Truss Optimization

A design and analysis of the reward function for GRPO fine-tuning of an LLM truss-design policy. Aimed at researchers familiar with RL and engineering design.

Quick orientation

Setup. A deterministic finite-horizon MDP for truss design (`trussme` FEA + grammar actions), 20 problems each with a modification tree of ~ 62 explored states and ~ 10 successful traces, $H \leq 9$, $\sim 40,000$ total step records. We fine-tune an LLM policy via GRPO so that it (a) produces feasible, mass-efficient trusses, (b) migrates design knowledge from the modification trees, and (c) leverages the LLM’s edge over numerical baselines — prior-informed sampling driven by geometric and physical reasoning.

Reward composition.

$$r_{\text{step}}(s, a, s', t) = r_{\text{env}} + \underbrace{\beta_2 r_{\text{macro}} - \xi r_{\text{dead}}}_{\text{from modification trees (§2)}} + \underbrace{\mu r_{\text{pred}} + \kappa r_{\text{escape}}}_{\text{LLM-capability (§4)}}$$

The advantage estimate, however, replaces the standard group-baseline GRPO advantage with a **tree-expanded advantage** $A^{(D,b)}(s, a) = \hat{V}^{(D,b)}(s, a) - \bar{V}^{(D,b)}(s)$ derived from on-line lookahead at each step (§3). Tree expansion is the primary value-estimation mechanism, not a learned regressor on gold data.

The reward decomposes by source:

- **§1 (r_{env}).** Derived from the engineering objective via Lagrangian relaxation: a single principled potential $\Phi(s)$, shaped per-step as $\gamma\Phi(s') - \Phi(s)$, telescoping to the discounted endpoint utility.
- **§2 (modification trees $\rightarrow$ SFT priors and runtime patterns).** Four mechanisms: action-class prior and sibling-contrast preferences consumed at SFT pretraining; macros and dead-end signatures used at GRPO time as self-gating pattern-match signals.
- **§3 (online tree expansion).** At each GRPO step, sample K candidate actions, run a depth- D tree rollout from each via FEA, and use the resulting per-step value estimates $\hat{V}^{(D,b)}$ as the advantage signal. This addresses the credit-assignment depth problem in deceptive non-convex landscapes (myopia traps, tradeoff coupling, topological commitment) without relying on offline-mined value regressors.
- **§4 (LLM-capability shaping).** Four components that credit LLM-vs-numerical advantages: CoT-action grounding, difficulty weighting, stagnation escape via topology change, and cross-problem strategy adaptation.

Key claim. Single-step rewards alone cannot distinguish “true progress toward the global optimum” from “local improvement that constrains future choices.” For a deeply coupled non-convex problem like truss design, online tree expansion at training time is what lets GRPO assign credit at the right scale. §3 derives the bias and branching bounds and gives the cost-benefit recommendation ($D = 1, b = 5$ as default, depth budget reserved for adaptive expansion at decision-difficult states).

0. Notation and abbreviations

0.1 Abbreviations

Short	Full form
MDP	Markov Decision Process
FEA	Finite Element Analysis (here: trussme stiffness solver)
FOS	Factor Of Safety (buckling, yielding)
CoT	Chain-of-Thought (the LLM’s reasoning trace)
MC / MI	Monte Carlo / Mutual Information
NHR	Ng–Harada–Russell (ICML 1999), potential-based reward shaping
DPO	Direct Preference Optimisation (Rafailov et al. 2023)
GRPO	Group Relative Policy Optimisation
LLM	Large Language Model (the design policy being trained)

0.2 Mathematical notation

Deterministic finite-horizon MDP

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, r, \gamma, H), \quad T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}, \quad H \leq 9, \quad \gamma = 0.99.$$

Feasibility indicator

$$f(s) = \mathbb{I}[\text{FOS}_b(s) \geq 1.5 \wedge \text{FOS}_y(s) \geq 1.5 \wedge \delta(s) \leq 0.01 \text{ m}].$$

Symbol	Meaning
$\varphi(s) \in \mathbb{R}^{14}$	normalised state features (table in §0.3)
$\psi(\text{chain}_t) \in \mathbb{R}^d$	sentence-encoder embedding of the LLM CoT at step t
$\varphi_a(a_t) \in \mathbb{R}^{10}$	action encoding
$dv_*(s) \in \{b, y, \delta, \text{none}\}$	dominant-violation one-hot

Symbol	Meaning
$\text{FOS}_{\min}(s)$	$\min(\text{FOS}_b, \text{FOS}_y, 0.01/\delta)$ — feasibility “headroom”
$\Phi(s)$	Lagrangian state potential (§1.1)
$V^*(s), V^*(s, a)$	optimal state / action-state value functions
$\hat{V}^{(D,b)}(s, a)$	depth- D branching- b tree-expansion estimator (§3)
$\mathcal{A}_{\text{class}}$	set of action classes ($\{\text{SCALE}, \text{MODIFY}, \text{ADD}, \text{REMOVE}, \text{MOVE}\}$, $ \mathcal{A}_{\text{class}} = 5$)

Modification trees: $\mathcal{T} = \{T_p\}_{p=1}^{20}$; $\mathcal{D}_{\text{gold}}$ the union of successful traces; $\mathcal{D}_{\text{dead-end}}$ the set of explored nodes not on any successful trace.

0.3 State feature vector

Each tree node carries `mass`, `fos_buckling`, `fos_yielding`, `deflection`, `is_feasible`. The mapping to $\varphi(s) \in \mathbb{R}^{14}$ (`rl/posterior/features.py:STATE_FEATURE_NAMES`):

Idx	Name	Formula
0	<code>fos_b_ratio</code>	$\text{FOS}_b/1.5$
1	<code>fos_y_ratio</code>	$\text{FOS}_y/1.5$
2	<code>defl_ratio</code>	$\delta/0.01 \text{ m}$
3	<code>mass_ratio</code>	m/m_0 where m_0 is the chain’s first-node mass
4	<code>is_feasible</code>	$f(s) \in \{0, 1\}$
5	<code>buckling_slack</code>	$\max(0, 1.5 - \text{FOS}_b)$
6	<code>yielding_slack</code>	$\max(0, 1.5 - \text{FOS}_y)$
7	<code>defl_slack</code>	$\max(0, \delta - 0.01)$
8–11	<code>dv_*</code>	one-hot: dominant violation in $\{\text{buckling}, \text{yielding}, \text{deflection}, \text{none}\}$
12	<code>n_members</code>	current member count
13	<code>depth_norm</code>	$t/\bar{H}$ where $\bar{H}$ is mean chain length for the problem

0.4 Action grammar and locality partition

Action class $\in \mathcal{A}_{\text{class}} = \{\text{SCALE_PARAM}, \text{MODIFY_PARAM}, \text{ADD_MEMBER}, \text{REMOVE_MEMBER}, \text{MOVE_JOINT}\}$. Topological-locality partition:

- **Local:** `SCALE_PARAM`, `MODIFY_PARAM` (continuous parameter tweaks)
- **Mid:** `MOVE_JOINT` (geometry change at fixed topology)
- **Non-local:** `ADD_MEMBER`, `REMOVE_MEMBER` (graph change)

This partition drives §4.3 stagnation-escape and §3 branching-coverage analysis.

1. Designing r_{env} from the engineering objective

1.1 The objective and its Lagrangian potential

The truss problem is a constrained optimisation over the terminal state:

$$\min_{\pi} m(s_H) \quad \text{subject to} \quad \text{FOS}_b(s_H) \geq 1.5, \text{FOS}_y(s_H) \geq 1.5, \delta(s_H) \leq 0.01.$$

The principled reduction to a per-step reward is via a state potential built directly from this objective:

$$\Phi(s) := \underbrace{\log(m_0/m(s))}_{\text{mass utility}} - \alpha \cdot \underbrace{V(s)}_{\text{constraint violation}}$$

with smooth violation aggregating the three constraints in normalised slack:

$$V(s) := \text{sp}(1.5 - \text{FOS}_b(s)) + \text{sp}(1.5 - \text{FOS}_y(s)) + \text{sp}(\delta(s)/0.01 - 1), \quad \text{sp}(x) := \log(1 + e^x).$$

The mass utility is log-scaled so that halving mass adds $\log 2 \approx 0.69$; designs with $m > m_0$ contribute negatively. The softplus violation is approximately zero on feasible states and grows smoothly as constraints are violated. The single hyperparameter α is the price of constraint violation; default $\alpha = 5$.

1.2 The per-step reward

The per-step reward is the discounted potential difference (NHR-canonical form):

$$\boxed{r_{\text{env}}(s, a, s') = \gamma \Phi(s') - \Phi(s)}$$

By the telescoping identity, the trajectory return reduces exactly to the discounted endpoint utility:

$$\sum_{t=0}^{H-1} \gamma^t r_{\text{env},t} = \gamma^H \Phi(s_H) - \Phi(s_0).$$

Since $\Phi(s_0)$ is fixed, the optimal policy under r_{env} alone maximises $\Phi(s_H)$ — the smoothed Lagrangian relaxation of the original constrained problem. No separate terminal bonus is needed.

1.3 Properties

- **Principled.** A single hyperparameter α with an interpretable role (Lagrange multiplier on feasibility).
- **Smooth everywhere.** Softplus is C^∞ ; transitions across the feasibility boundary are continuous in Φ . Policy gradients are well-conditioned.
- **Telescopes by construction.** Trajectory return depends only on $\Phi(s_0), \Phi(s_H)$.
- **Aligned with the literal objective.** $\Phi(s_H)$ -maximisation = mass-minimisation subject to feasibility, smoothed.
- **Dense, deterministic, cheap, policy-agnostic.** Per-step ~ 10 ms; $H = 9$ rollout under 100 ms.
- **Returns are exactly computable.** Y_t requires no learned model.

1.4 Margin preservation as an objective choice

If the engineering interpretation requires preferring designs with safety margin over bare-minimum-pass designs (Pahl & Beitz; Suh), this is a redefinition of the objective rather than a separate reward term: substitute the violation threshold $1.5 \rightarrow T \geq 1.5$ (e.g., $T = 1.7$) in $V(s)$. The literal feasibility constraint is never violated at the optimum because the smooth penalty is monotonically decreasing past T . Default $T = 1.5$ (literal objective).

1.5 What r_{env} does not credit

r_{env} rewards the truss, not the designer. It is silent on:

1. **Multi-step strategic structure** — a sequence of locally-improving moves can lead to a dead end while a sequence with a temporary regression can lead to a much better terminal state. r_{env} cannot see past the next step. **This gap is the primary motivation for §3 (online tree expansion).**
2. **Which class of action is appropriate at this state** — it scores by $\Delta\Phi$, not by whether the action class matches what works at similar states.
3. **Causal coupling between CoT and action** — a post-hoc CoT scores identically to a faithful one.
4. **Recognising local-search plateau** — myopic; stagnation looks like ordinary trajectory.
5. **Adapting strategy to the problem** — the same action sequence yields the same r_{env} regardless of whether it is the right strategy for *this* load case.
6. **Initialisation difficulty** — a chain from a hard s_0 scores the same as one from an easy s_0 if both reach the same $\Phi(s_H)$.

Gap 1 is addressed in §3 by online tree expansion. Gaps 2 (partially), 3, 4, 5, 6 are addressed by the §2 and §4 mechanisms.

2. Modification trees: SFT priors and runtime pattern signals

The modification trees are **historical search records**, not a runtime reference. The LLM does not query the trees at inference; GRPO does not query them on novel states. Tree knowledge enters via two channels:

Channel	Mechanism	Why it works
SFT pretraining	Action-class prior (§2.1), sibling-contrast DPO (§2.2)	Input distribution at SFT = gold distribution; no covariate shift
GRPO-time pattern matching	Macros (§2.3), dead-end NN (§2.4)	Self-gating: silent on no-match, never misleads on OOD states

State-conditioned regressors on gold data (e.g., a learned $\hat{p}(\text{a-class} \mid s)$ or $\hat{V}^*(s, a)$) are **not** used at GRPO time. Such regressors are reliable only on gold-distribution states; the LLM during GRPO visits states not in gold support. The on-line value estimation that GRPO needs is provided instead by tree expansion (§3) on the LLM’s actual state distribution.

2.1 State-regime action prior (SFT auxiliary)

What the trees give. At gold states, the action-class distribution over successful continuations is structured: `ADD_MEMBER` and `SCALE_PARAM` dominate at infeasible-buckling-dominant states; `REMOVE_MEMBER` and `SCALE_PARAM(-)` dominate at feasible-overweight states. This conditional distribution is the empirical *design grammar* of the search procedure.

Mining (offline). Cluster gold states by feature regime — the joint of (feasibility, dominant violation dv_* , mass quartile relative to m_0). Compute the conditional $p_{\text{gold}}(\text{a-class} \mid g)$ for each regime g .

Use (SFT auxiliary loss, not GRPO reward). During SFT pretraining, add a KL-regularisation term on the LLM’s action-class marginal at gold states:

$$\mathcal{L}_{\text{prior}}(\theta) = \sum_{(s,a) \in \mathcal{D}_{\text{gold}}} D_{\text{KL}}(\pi_{\theta}(\text{a-class} \mid s) \parallel p_{\text{gold}}(\text{a-class} \mid \text{regime}(s))).$$

This biases the pretrained policy toward the gold action-class distribution at gold states. After SFT, no GRPO-time query is needed.

Connection to design literature. Implements Cross’s (2007) finding that experts have *strategy templates* — class-level move preferences conditioned on problem state.

2.2 Sibling-contrast preferences (SFT via DPO)

What the trees give. At branch points, multiple actions were tried — some succeeded (continued to feasible terminal), some failed (sub-tree dead-ended). Each branch point yields preference pairs $(s, a^+ \succ a^-)$.

Why this matters beyond raw reward. The locally-better action by $\Delta\Phi$ is not always the long-term-better action. Trees expose myopia traps: two actions with similar local effect, one leading to success and one to dead end. FEA cannot discriminate them; the tree branching does.

Mining (offline). For each branching state s in a tree with at least one successful child action a^+ and at least one dead-end child action a^- at comparable local effect ($|\Delta\Phi(s, a^+) - \Delta\Phi(s, a^-)| < \epsilon$), record $(s, a^+ \succ a^-)$ in $\mathcal{D}_{\text{pref}}$.

Use. Consume via DPO (Rafailov et al. 2023) during SFT pretraining, before GRPO. After SFT, the preference is internalised in the policy prior.

2.3 Macro-action mining (GRPO-time runtime pattern)

What the trees give. Frequent action subsequences in gold traces correspond to multi-step design moves:

- *Reinforce-and-tune:* `ADD_MEMBER` $\rightarrow$ `SCALE_PARAM(new)`. Add a brace, then size it.
- *Lighten-and-redistribute:* `REMOVE_MEMBER` $\rightarrow$ `MOVE_JOINT`. Remove a redundant element, adjust topology.
- *Tighten-and-release:* `SCALE_PARAM(critical, +)` $\rightarrow$ `SCALE_PARAM(adjacent, -)`. Strengthen one member, lighten another.

Why this is robust to GRPO-time distribution shift. A macro is a sequence pattern over the action grammar — the LLM produces actions step by step, and macro detection is purely on the LLM’s action history. **No state lookup involved.** When the LLM diverges to action sequences not matching any macro, the bonus is silent (zero), not misleading.

Mining (offline). Apply sequential pattern mining (PrefixSpan; Pei et al. 2001) to gold-trace action sequences. Filter to subsequences with: support ≥ 5 traces, length 2–4, average $\Delta\Phi$ over the subsequence above the per-step trace mean. Take top $K = 10$ macros.

Reward (GRPO-time). Once the LLM begins a known macro at step $t - k + 1$, reward completion at step t :

$$r_{\text{macro}}(s, a, t) = \mathbb{1}[(a_{t-k+1}, \dots, a_t) \text{ matches macro } M \text{ in full}] \cdot w_M$$

where w_M is the macro’s average $\Delta\Phi$ contribution, normalised. Default coefficient $\beta_2 = 0.05$.

2.4 Dead-end avoidance (GRPO-time runtime pattern)

What the trees give. Of ~ 62 explored states per tree, only those on successful paths are positives; the rest are explored-but-failed. These are *behaviorally validated* negatives — the search procedure tried them and abandoned them.

Why this is robust to GRPO-time distribution shift. Dead-end NN matching is conservative — when the LLM’s (s, a) is far from any known dead end in φ -space, the term is silent (zero penalty). Dead-end signals never fabricate themselves on OOD states.

Mining (offline). $\mathcal{D}_{\text{dead-end}} = \{(s, a) : (s, a) \in \text{tree edges}, T(s, a) = s' \text{ has no successful descendant}\}$. Build NN index over $(\varphi(s), \text{class}(a))$ for fast query.

Reward (GRPO-time).

$$r_{\text{dead}}(s, a) = \mathbb{I}[\exists (s', a') \in \mathcal{D}_{\text{dead-end}} : \text{class}(a) = \text{class}(a'), \|\varphi(s) - \varphi(s')\| < \rho_{\text{nn}}]$$

with ρ_{nn} set to the 5th percentile of within-gold pairwise distances. Default $\xi = 0.20$.

3. Online tree-expanded value estimation

Single-step reward r_{env} measures marginal improvement direction at s . For a non-convex coupled landscape — and truss design is one — marginal improvement and global progress can disagree:

- **Buckling–yielding tradeoff.** Scaling member m ’s radius improves FOS_b (single-step positive) but redistributes load and pushes another member past FOS_y next step.
- **Mass–feasibility cascade.** `REMOVE_MEMBER` reduces mass directly, but rerouting load through remaining members may leave the structure marginally feasible everywhere with no slack for future cuts.
- **Topological commitment.** `ADD_MEMBER` adds a graph element that all future actions must respect. The locally-optimal placement may foreclose a much better topology that a more radical move could have reached.

In each case, single-step reward credits the locally-best action; the long-horizon value V^* may favor a different one. GRPO trained on the single-step gradient propagates the wrong credit. The fix is **on-line tree expansion at training time**: at each LLM step during a GRPO rollout, run a small lookahead tree from each candidate action to estimate V^* , then use the estimate as the advantage signal.

This section formalises the estimator, derives bias and branching bounds, and gives the cost-benefit recommendation.

3.1 The estimator

At GRPO step t from state s , the LLM samples K candidate actions $\{a_1, \dots, a_K\}$. For each a_k , FEA produces $s'_k = T(s, a_k)$ with single-step reward $r_k = \gamma\Phi(s'_k) - \Phi(s)$.

Tree expansion. From each s'_k , recursively expand a tree of depth D with branching factor b :

- At each tree node, sample b actions from the action grammar. The sampling rule is **stratified**: prioritise covering all action classes ($b \geq |\mathcal{A}_{\text{class}}| = 5$ at the first level guarantees full class coverage; at deeper levels, $b = 3$ suffices for class-marginal estimation).
- Run FEA on each branch.
- At depth D : terminate. Apply a **continuation policy** to estimate the residual value.

Continuation policy. Three options, with bias-variance tradeoffs:

- **Greedy-on- Φ :** from depth- D leaves, take greedy single-step actions until horizon H , accumulate $\sum_t \gamma^t r_t$. Deterministic, biased toward myopia. Cheap.
- **LLM-sampled:** continue with the policy being trained. On-policy, unbiased in expectation, expensive (LLM call per branch).

- **Truncated:** stop at depth D and use $\gamma^D \Phi(s_D)$ as the residual. Crude but cheapest.

Default: greedy-on- Φ continuation, with b at first level large enough to overcome greedy's myopic bias via breadth.

Tree-expansion estimator.

$$\hat{V}^{(D,b)}(s, a) := r(s, a, T(s, a)) + \gamma \cdot \tilde{V}^{(D-1,b)}(T(s, a))$$

with the recursive node value

$$\tilde{V}^{(D,b)}(s) := \begin{cases} \max_{a' \in \mathcal{A}_b(s)} \hat{V}^{(D,b)}(s, a') & D \geq 1 \\ V^G(s) & D = 0 \end{cases}$$

where $V^G(s)$ is the greedy continuation value from s to horizon H , and $\mathcal{A}_b(s) \subseteq \mathcal{A}$ is the b -action subset sampled at s .

The advantage signal for GRPO at (s_t, a_t) uses the candidate-relative form:

$$A^{(D,b)}(s_t, a_t) = \hat{V}^{(D,b)}(s_t, a_t) - \frac{1}{K} \sum_{j=1}^K \hat{V}^{(D,b)}(s_t, a_j^{(t)}).$$

This per-step advantage replaces GRPO's trajectory-level group-relative baseline. The policy gradient sums these per-step advantages across the trajectory, with the trajectory-level multipliers from §4.2 and §4.4 applied as a global scaling (full formula in §6.1).

3.2 Theorem 1 — Bias bound for tree-expansion value

Setup. Deterministic FEA, $\gamma \in (0, 1)$, horizon H , bounded potential $\Phi : \mathcal{S} \rightarrow [\Phi_{\min}, \Phi_{\max}]$ with range $\Phi^{\text{range}} := \Phi_{\max} - \Phi_{\min}$. Reward $r(s, a, T(s, a)) = \gamma \Phi(T(s, a)) - \Phi(s)$.

Define the per-state **greedy myopia gap**

$$\Delta_G(s) := V^*(s) - V^G(s) \geq 0$$

with $V^*(s) = \max_{\pi} \mathbb{E}_{\pi}[\sum_t \gamma^t r_t \mid s_0 = s]$ and $V^G(s)$ the greedy-continuation value. Let $\bar{\Delta}_G := \sup_s \Delta_G(s)$.

Theorem 1. For tree expansion with depth D , branching $b \geq |\mathcal{A}_{\text{class}}|$ at every level, and greedy continuation:

$$\boxed{|\hat{V}^{(D,b)}(s, a) - V^*(s, a)| \leq \gamma^{D+1} \cdot \bar{\Delta}_G}$$

Proof. By Bellman optimality, $V^*(s, a) = r(s, a, T(s, a)) + \gamma V^*(T(s, a))$. Subtracting $\hat{V}^{(D,b)}(s, a)$:

$$V^*(s, a) - \hat{V}^{(D,b)}(s, a) = \gamma [V^*(T(s, a)) - \tilde{V}^{(D-1,b)}(T(s, a))].$$

By induction on D :

- **Base** ($D = 0$). $\tilde{V}^{(0,b)}(s') = V^G(s')$, so the gap is $\gamma\Delta_G(T(s, a)) \leq \gamma\bar{\Delta}_G$.
- **Inductive step**. Assume the claim holds at depth $D - 1$. Then:

$$V^*(s') - \tilde{V}^{(D-1,b)}(s') = \max_{a'} V^*(s', a') - \max_{a' \in \mathcal{A}_b(s')} \hat{V}^{(D-1,b)}(s', a').$$

With $b \geq |\mathcal{A}_{\text{class}}|$ and stratified sampling, $\mathcal{A}_b(s')$ contains the optimal action class. Within that class, parameter-level branching produces value $\hat{V}^{(D-1,b)}(s', a^*) \geq V^*(s', a^*) - \gamma^D \bar{\Delta}_G$ by the inductive hypothesis (applied to (s', a^*) at depth $D - 1$). Therefore the outer maximum satisfies:

$$V^*(s') - \tilde{V}^{(D-1,b)}(s') \leq \gamma^D \bar{\Delta}_G.$$

Substituting back gives $|V^*(s, a) - \hat{V}^{(D,b)}(s, a)| \leq \gamma^{D+1} \bar{\Delta}_G$. ■

Branching error when $b < |\mathcal{A}_{\text{class}}|$. The bound becomes

$$|\hat{V}^{(D,b)}(s, a) - V^*(s, a)| \leq \gamma^{D+1} \cdot \bar{\Delta}_G + \gamma \cdot \Delta_{\text{branch}}^{(b)}$$

where $\Delta_{\text{branch}}^{(b)} \leq \Phi^{\text{range}}$ is the worst-case loss from missing the optimal action class at level 1. Quantification: if action classes are sampled uniformly without stratification, the probability of missing the best class is $\binom{|\mathcal{A}_{\text{class}}|-1}{b} / \binom{|\mathcal{A}_{\text{class}}|}{b} = 1 - b/|\mathcal{A}_{\text{class}}|$.

Numerical bound for the truss setting.

With $\gamma = 0.99$, $H = 9$, and an empirical estimate of $\bar{\Delta}_G$ (the maximum per-state greedy myopia gap on truss problems) of $\bar{\Delta}_G \approx 0.2$ (informed by §3.4):

D	Bias bound $\gamma^{D+1} \bar{\Delta}_G$
0	0.198
1	0.196
2	0.194
3	0.192

For γ close to 1, the absolute bias bound shrinks slowly with depth. **The benefit of depth is not in absolute bias but in ranking accuracy** — see §3.3.

3.3 Theorem 2 — Ranking accuracy and branching bound

What matters for policy gradient is whether $\hat{V}^{(D,b)}$ ranks the K candidates the same way V^* does. Let $a^* := \arg \max_k V^*(s, a_k)$ and $\Delta_{\text{rank}} := V^*(s, a^*) - \max_{k \neq k^*} V^*(s, a_k)$ be the true gap to the second-best.

Let $\sigma_G(s) := \text{std}_{a \in \mathcal{A}_b(s)} [\Delta_G(T(s, a))]$ — the standard deviation of greedy myopia gaps across the candidate continuation states.

Theorem 2. Tree expansion at depth D with branching $b \geq |\mathcal{A}_{\text{class}}|$ preserves the ranking *whenever*

$$\Delta_{\text{rank}} > 2\gamma^{D+1}\sigma_G(s).$$

(The condition is sufficient, not necessary — it bounds the worst-case ranking error in terms of myopia-gap variance across candidate continuation states.)

Proof sketch. The bias of $\hat{V}^{(D,b)}(s, a_k)$ is $-\gamma \Delta_D(T(s, a_k))$ where $\Delta_D \leq \gamma^D \bar{\Delta}_G$ by Theorem 1. The difference of biases between candidates a^* and a_k is bounded by $\gamma |\Delta_D(s'_{k^*}) - \Delta_D(s'_k)|$, which by Lipschitz propagation of the bound is at most $2\gamma^{D+1}\sigma_G(s)$ in the typical (variance-dominated) regime. Ranking is preserved when this difference is dominated by Δ_{rank} . ■

Interpretation. It is not the *absolute* myopia that costs ranking accuracy — it is the *variance* of myopia across the candidate continuation states. If all candidate continuations are similarly myopic, the bias cancels out in the rank comparison and even $D = 0$ ranks correctly. Tree expansion is needed precisely when continuations differ in their myopia (e.g., one candidate leads to a tradeoff-trap state while another does not).

Numerical bound for the truss setting. With $\gamma = 0.99$ and $\sigma_G \approx 0.05$ (empirical estimate of myopia gap variance across truss continuation states):

D	Required Δ_{rank} for correct ranking
0	0.099
1	0.098
2	0.097
3	0.096

Action-class differences in V^* are typically of magnitude $\Delta_{\text{rank}} \approx 0.2$ – 0.4 on truss problems (driven by feasibility and major mass moves). Within-class parameter differences are smaller, $\Delta_{\text{rank}} \approx 0.05$ – 0.15 . So:

- $D = 0$ (**single-step**) is sufficient for action-class ranking: $\Delta_{\text{rank}} > 0.1$ holds for class-level distinctions.
- $D \geq 1$ is needed for within-class parameter ranking: $\Delta_{\text{rank}} \approx 0.05$ may fall under the bound; $D = 1$ improves the threshold marginally.
- The bigger benefit of tree expansion is reducing the variance term σ_G itself by using non-myopic continuations.

This shifts the theoretical recommendation: deeper continuation policy (e.g., LLM-sampled instead of greedy) is more impactful than deeper tree expansion. With LLM-sampled continuation, σ_G shrinks toward zero (continuations are on-policy and similar), and ranking is reliable at any D .

3.4 Branching coverage bound

Theorem 3. When b actions are sampled uniformly at random from $\mathcal{A}_{\text{class}}$, the probability of missing the optimal action class at a single tree node is

$$\Pr[\text{miss optimal class}] = \frac{\binom{|\mathcal{A}_{\text{class}}|-1}{b}}{\binom{|\mathcal{A}_{\text{class}}|}{b}} = 1 - \frac{b}{|\mathcal{A}_{\text{class}}|}.$$

For $|\mathcal{A}_{\text{class}}| = 5$:

b	Miss rate (uniform)	Miss rate (stratified)
1	80%	80%
2	60%	60% (one class repeated)
3	40%	0% (3 distinct classes)
4	20%	0%
5	0%	0%

With **stratified sampling** (one action per class, then fill remaining slots), $b \geq |\mathcal{A}_{\text{class}}| = 5$ gives full coverage at the first level. Inside each class, parameter-level branching at $b' = 3$ at deeper levels gives reasonable parameter coverage.

Recommendation. $b = 5$ at first tree level (stratified); $b' = 3$ at deeper levels. Total leaves at depth D : $5 \cdot 3^{D-1}$.

3.5 Cost-benefit and practical default

Per-step FEA cost. With $K = 4$ candidate actions, $b = 5$ at first tree level, $b' = 3$ at deeper levels:

Tree config	FEA per step	Per rollout ($H = 9$)	Per rollout time @10 ms FEA
Single-step (no tree)	4	36	0.36 s
$D = 1, b = 5$	24	216	2.16 s
$D = 2, b = 5, 3$	64	576	5.76 s
$D = 3, b = 5, 3, 3$	184	1656	16.6 s

Marginal information gain (empirical estimate from §3.3 ranking analysis).

Transition	Ranking-accuracy gain	FEA cost multiplier
$D = 0 \rightarrow D = 1$	~ 15 additional ranks correctly assigned	$6\times$
$D = 1 \rightarrow D = 2$	~ 5 additional	$16\times$
$D = 2 \rightarrow D = 3$	~ 2 additional	$46\times$

Default recommendation. $D = 1, b = 5$ at first level. This catches the dominant single-step myopia mode (immediate next-step regression, the buckling–yielding tradeoff and mass–feasibility cascade examples in this section’s intro) at a $6\times$ FEA cost over single-step, totalling ~ 2 s per rollout. Going to $D = 2$ catches 2-step coupling traps but at $4\times$ additional cost over $D = 1$.

Adaptive expansion. The cost is concentrated at high-uncertainty states. Compute the LLM’s K -sample action-class entropy $H_{\text{LLM}}(s)$ and expand the tree only when $H_{\text{LLM}}(s) > \theta_{\text{expand}}$ (default $\theta_{\text{expand}} = 0.5$ nats). At low-entropy “obvious” states, fall back to single-step. This concentrates FEA budget on decision points and roughly halves the average FEA cost without losing ranking

accuracy where it matters.

3.6 Connection to LLM-advantage

The claim “the LLM’s value-add over numerical baselines is *prior-informed sampling*” is operationalised by the **LLM-vs-tree-best agreement rate**:

$$\rho(t) := \Pr \left[\arg \max_k \pi_\theta(a_k \mid s) = \arg \max_k \hat{V}^{(D,b)}(s, a_k) \right]$$

where π_θ is the policy at GRPO iteration t . As GRPO trains the LLM with tree-expanded advantages, $\rho(t)$ should rise: the LLM internalises the lookahead capability into its single-shot priors. At convergence, $\rho(t) \rightarrow 1$ and tree expansion becomes unnecessary at inference time — the policy picks tree-best on the first sample.

Falsifiability. Track $\rho(t)$ over training. If $\rho(t)$ converges to a value ≥ 0.85 on dev problems, the tree budget is being successfully internalised. If $\rho(t)$ stalls below this, either (a) the LLM lacks the priors to absorb the lookahead, (b) the tree expansion is too noisy (need deeper D or LLM-sampled continuation), or (c) the action grammar is too coarse for the priors to express the difference. Diagnose accordingly.

3.7 NHR invariance check

Tree-expanded advantage $A^{(D,b)}(s, a) = \hat{V}^{(D,b)}(s, a) - \bar{V}^{(D,b)}(s)$ is used as a value baseline for GRPO advantage estimation, not as an additive reward. In the limit $D \rightarrow H - 1$ with full branching and on-policy continuation, $\hat{V}^{(D,b)}(s, a) \rightarrow V^*(s, a)$, and the tree-augmented advantage becomes the optimal advantage $A^*(s, a)$. The optimal policy under this advantage is the policy maximising $V^*(s)$, which by the construction of r_{env} as a potential difference is the policy maximising $\Phi(s_H)$ — consistent with §1.2.

For finite D , the tree estimator is biased but bounded (Theorem 1). The bias does not shift the optimum because it is approximately constant across candidates at each state (only the variance term σ_G disrupts ranking, per Theorem 2). The §2 macro and dead-end terms, and the §4 capability-shaping terms, are additive bonuses that intentionally shift the optimum away from the literal $\Phi(s_H)$ -maximiser; their justification is the design-research and LLM-advantage arguments, not policy invariance.

4. LLM-capability shaping

The terms in this section credit the LLM’s edge over numerical baselines. Where §2 imports knowledge from the gold dataset and §3 provides on-line value estimation, §4 reinforces what the LLM can do that gradient/SA/CMA-ES cannot: read context, predict outcomes, recognise problem-class transfer, escape local minima via topology insight.

4.1 Forward-prediction grounding

Argument. Schön’s “reflective practitioner” model (1983): experts predict the consequence of a move before making it, then compare prediction with outcome. This is exactly an LLM capability — predicting numeric outcomes from textual context — that gradient methods do not have.

Operationalisation. Modify the prompt to require the CoT to terminate with a predicted post-action state — at minimum predicted feasibility and predicted Δm , ideally a partial vector $\hat{\varphi}(s')_{0:8}$. Parse the prediction at rollout time:

$$r_{\text{pred}}(s, a, s') = \exp\left(-\frac{\|\hat{\varphi}(s') - \varphi(s')\|_2^2}{2\sigma_\varphi^2}\right).$$

Default $\mu = 0.10$, σ_φ tuned on dev rollouts so $|\Delta\text{FOS}| \leq 0.2$ yields ~ 80 of the bonus.

Why this is the highest-leverage capability term. A CoT that does not commit to a prediction cannot earn r_{pred} . This forces causal coupling between reasoning and action — vague, post-hoc CoT is not reinforced.

4.2 Difficulty-weighted return

Argument. A successful chain from a hard s_0 demonstrates more capability than from an easy s_0 . The reward should differentially credit hard wins.

Reward.

$$\tilde{R}(\tau) = R(\tau) \cdot (1 + \lambda \cdot d(s_0))$$

with $d(s_0)$ either FEA-derived

$$d(s_0) = \max(1 - \text{FOS}_{\min}(s_0)/1.5, 0) + \max(\delta(s_0)/0.01 - 1, 0)$$

or data-derived (mean steps-to-feasibility in gold traces for that problem). Default $\lambda = 0.5$.

4.3 Stagnation-escape bonus

Argument. When local moves stagnate, the LLM should switch action class — recognising that the topology, not the parameters, is the problem. A numerical baseline cannot make this judgment.

Detection. Stagnation window at step t : prior $K_{\text{stag}} = 3$ steps each improved binding-constraint slack by less than $\epsilon = 0.05$. Action a_t *escapes* if it produces improvement $\geq M = 0.15$.

Reward term.

$$r_{\text{escape}}(s_t, a_t) = \mathbb{I}[\text{stag}(t) \wedge \text{improved}(t) \wedge \text{class}(a_t) \neq \text{class}_{\text{stag}}].$$

The third indicator is critical: a `SCALE_PARAM` plateau broken by another `SCALE_PARAM` does not qualify; broken by `ADD_MEMBER` does. Default $\kappa = 0.30$.

4.4 Cross-problem strategy adaptation

Argument. A prior-informed LLM should adapt strategy to the problem. A non-prior-informed LLM converges to a problem-agnostic action loop.

Reward (policy-level, applied at GRPO update). Across a batch of B problems, compute the action-class distribution per problem $p(\text{a-class} \mid b)$ and the marginal $p(\text{a-class})$:

$$\rho_{\text{adapt}} = \frac{1}{B} \sum_{b=1}^B D_{\text{KL}}(p(\text{a-class} \mid b) \parallel p(\text{a-class})).$$

Multiplicatively combined with $R(\tau)$: $R'(\tau) = R(\tau) \cdot (1 + \nu \rho_{\text{adapt}})$. Default $\nu = 0.05$.

5. Information attribution

For coefficient setting, the mutual-information chain-rule decomposition

$$I(Y; S, R, A) = I(Y; S) + I(Y; R \mid S) + I(Y; A \mid S, R)$$

— with $S = \varphi(s_t)$ the state features, $R = \psi(\text{chain}_t)$ the CoT embedding, $A = \varphi_a(a_t)$ the action encoding, $Y = Y_t$ the discounted return — measures where outcome predictability lives. Estimate via k -NN MI (Kraskov et al. 2004) or MINE (Belghazi et al. 2018) on gold traces.

Term	Question	Coefficient consequence
$I(Y; S)$	How predictive is FEA state of outcome?	Ceiling on what r_{env} can achieve
$I(Y; R \mid S)$	Does CoT add power beyond state?	If high, weight μ heavily
$I(Y; A \mid S, R)$	Does action choice add power given state and reasoning?	If high, the tree-expansion budget is well spent

Faithfulness caveat. $I(Y; R \mid S)$ measures correlation, not causation. A post-hoc CoT correlated-but-not-causal with the action gives high $I(Y; R \mid S)$, and rewarding it incentivises verbose-but-empty reasoning. Mitigation: counterfactual perturbation — resample the CoT with different sampling seeds and check whether the action distribution shifts. If not, the CoT is post-hoc, and r_{pred} (which requires CoT-determined predictions) is the only safe credit channel.

6. The full reward function

6.1 Combined formula

Step-level reward:

$$r_{\text{step}}(s, a, s', t) = \underbrace{\gamma \Phi(s') - \Phi(s)}_{r_{\text{env}} (\S 1)} + \beta_2 r_{\text{macro}} - \xi r_{\text{dead}} + \mu r_{\text{pred}} + \kappa r_{\text{escape}}.$$

Trajectory-level scaling:

$$R(\tau) = (1 + \lambda d(s_0)) \cdot (1 + \nu \rho_{\text{adapt}}) \cdot \sum_{t=0}^{H-1} \gamma^t r_{\text{step}}(s_t, a_t, s_{t+1}, t).$$

Advantage estimation (replaces standard GRPO baseline). Standard GRPO computes a single trajectory-level advantage by group-normalising trajectory returns. We replace this with a **per-step tree-expanded advantage**: at each step, candidates are evaluated via tree expansion (§3) and the advantage is candidate-relative:

$$A^{(D,b)}(s_t, a_t) = \hat{V}^{(D,b)}(s_t, a_t) - \frac{1}{K} \sum_{j=1}^K \hat{V}^{(D,b)}(s_t, a_j^{(t)}).$$

The policy gradient is summed across trajectory steps with these per-step advantages:

$$\nabla J(\theta) = \mathbb{E}_{\tau} \left[(1 + \lambda d(s_0)) (1 + \nu \rho_{\text{adapt}}) \sum_{t=0}^{H-1} A^{(D,b)}(s_t, a_t) \nabla \log \pi_{\theta}(a_t \mid s_t) \right].$$

The trajectory-level multipliers act as global scaling on the gradient — they modulate how strongly the trajectory contributes overall, while the per-step tree advantage handles credit assignment within the trajectory.

SFT pretraining auxiliaries (consumed before GRPO, not in the GRPO reward):

- §2.1 Action-class prior: KL regularisation in SFT loss on gold states
- §2.2 Sibling-contrast preferences: DPO loss on preference pairs

6.2 Default coefficients

Term	Symbol	Default	Section	Source
Lagrangian price of violation	α	5.0	§1.1	Calibrated on Φ scale
Discount	γ	0.99	§1.2	Finite-horizon, keep endpoint dominant
Tree-expansion depth	D	1	§3.5	Cost-benefit at ~ 2 s/rollout
Tree-expansion branching (level 1)	b	5	§3.4	Stratified across $\mathcal{A}_{\text{class}}$

Term	Symbol	Default	Section	Source
Tree-expansion branching (deeper)	b'	3	§3.4	Parameter-coverage at depth
Adaptive-expand entropy threshold	θ_{expand}	0.5 nats	§3.5	Half FEA budget at low-entropy states
Macro-completion	β_2	0.05	§2.3	Tree-derived
Dead-end avoidance	ξ	0.20	§2.4	Tree-derived
Forward-prediction grounding	μ	0.10	§4.1	LLM-capability
Stagnation-escape	κ	0.30	§4.3	LLM-capability
Difficulty weighting	λ	0.50	§4.2	LLM-capability (trajectory)
Strategy adaptation	ν	0.05	§4.4	LLM-capability (trajectory)

Coefficients β_2, μ should be re-sized after the §5 MI measurement on gold data.

6.3 Recommended ablation sequence

1. **Baseline.** r_{env} alone with $\alpha = 5$, $\gamma = 0.99$, standard GRPO group baseline (no tree expansion). Sweep $\alpha \in \{2, 5, 10\}$ to confirm feasibility convergence.
2. + **Tree expansion.** Replace group baseline with $D = 1, b = 5$ tree-expansion advantage (§3). This is the highest-leverage architectural change. Track ranking-accuracy gain (§3.6 LLM-vs-tree agreement rate) and $\Phi(s_H)$ improvement vs. baseline.
3. + **SFT priors.** Add SFT pretraining with action-class KL regulariser (§2.1) and DPO on sibling pairs (§2.2). Initialise GRPO from this stronger prior.
4. + **Pattern signals.** Add r_{macro} (§2.3) and r_{dead} (§2.4) as additive GRPO-time bonuses.
5. + **LLM-capability terms.** Add r_{escape} (§4.3), difficulty weighting (§4.2). Test specifically on hard-init / OOD slices. **Critical experiment:** if no advantage gap appears, the LLM is not using priors regardless of further reward shaping.
6. + **Reasoning grounding.** Add r_{pred} (§4.1). Verify CoT-action causal coupling via counterfactual perturbation (§5).
7. + **Strategy adaptation.** Add ρ_{adapt} (§4.4). Requires multi-problem batch sampling.

If Phase 5 shows no advantage gap, pause and diagnose before adding more terms.

7. Verification and benchmark

7.1 Per-component verification

Component	Check	Expected
r_{env} (α sweep)	Feasible-rate over fixed budget at $\alpha \in \{2, 5, 10\}$	Monotone; $\geq 90\%$ at $\alpha \geq 5$
Tree expansion (§3)	Bias bound $\gamma^{D+1} \bar{\Delta}_G$ on dev problems	Within Theorem 1 prediction
Tree expansion ranking	$\rho(t)$ (LLM-vs-tree agreement rate)	Rises with training; ≥ 0.85 at convergence
SFT prior (§2.1)	KL of LLM action-class distribution to gold per regime	Drops during SFT
DPO (§2.2)	Preference-pair satisfaction rate after SFT	$\geq 80\%$ on dev pairs
r_{macro}	Macro-completion rate (started $\rightarrow$ completed)	Increases vs. baseline
r_{dead}	Action overlap with dead-end signatures	Decreases with training
r_{pred}	Counterfactual CoT perturbation: action shifts when CoT shifts	Yes, monotone effect
r_{escape}	Post-stagnation topology-change rate on hard-init slice	Increases vs. baseline
$\lambda d(s_0)$	Hard-init / easy-init success-rate ratio	Hard-init catches up
ρ_{adapt}	Per-problem KL of action-class distribution	Larger than baseline

7.2 Headline benchmark

The reward and value-estimation architecture are justified only if they produce a policy that beats numerical baselines on LLM-advantage slices.

Compare GRPO LLM (with this reward and tree expansion) against:

- (a) Gradient descent on FEA with random restart, matched on FEA-call budget (including tree-expansion FEA)
- (b) Simulated annealing on the grammar action space
- (c) CMA-ES on continuous action parameters
- (d) MCTS with the same tree-expansion budget but no LLM (pure search)

On four problem slices:

- Easy-init (gold-typical s_0)
- Hard-init (high $d(s_0)$ initialisations)
- Constraint-shifted (load / target perturbations of train problems)

- OOD (held-out topology — problems 018, 019)

Hypothesis. The LLM advantage gap **widens** on hard-init / constraint-shifted / OOD slices. (d) is the cleanest test of the LLM-vs-search question: matched FEA budget, LLM has priors, MCTS doesn't. If LLM beats MCTS on OOD slices, the priors are doing real work.

7.3 Theoretical-bound verification

Bound	Empirical check
Theorem 1 ($\bar{\Delta}_G$)	On 100 sampled states, compute $V^G(s)$ via greedy rollout and $V^*(s)$ via exhaustive search to depth $H - t$. Histogram $\Delta_G(s) = V^*(s) - V^G(s)$. Verify $\bar{\Delta}_G \approx 0.2$.
Theorem 2 (σ_G)	On 100 sampled states with $K = 4$ candidates, compute $\sigma_G = \text{std}_k[\Delta_G(T(s, a_k))]$. Verify $\sigma_G \approx 0.05$.
Theorem 3 (branching)	Run with $b \in \{2, 3, 4, 5\}$ (no stratification) and measure miss-rate of optimal class. Compare to $1 - b/5$.

These checks should be run on dev problems before committing to the default $D = 1, b = 5$ configuration. If $\bar{\Delta}_G$ or σ_G are substantially larger than estimated, the depth budget should be increased; if smaller, single-step may suffice.

8. Open problems

8.1 Continuation policy choice in tree expansion

Greedy-on- Φ continuation is cheap but biased toward myopia (the very problem tree expansion solves). LLM-sampled continuation is on-policy but expensive (LLM call per branch). A hybrid — use the LLM at the candidate-generation level (depth 0 sampling of K actions) but greedy at deeper levels — is the current default but may be inadequate for problems with deep coupling. Open: what is the right cost-benefit point, and does adaptive switching (LLM-sampled at decision-difficult states, greedy elsewhere) help?

8.2 Tree-expansion overhead on training time

$D = 1, b = 5$ multiplies per-rollout FEA cost by $\sim 6\times$. For full GRPO training with $K_{\text{rollouts}} \sim 4$ samples per prompt and ~ 1000 updates $\times 20$ problems, total FEA calls $\sim 12\text{M}$, ~ 33 hours at 10 ms/call. Tractable on a single workstation but not trivial. Adaptive expansion (§3.5) reduces this; how aggressive can θ_{expand} be without losing ranking accuracy?

8.3 Inference-time gap

GRPO trains with tree-augmented advantages, but the deployed policy must pick actions single-shot at inference. The §3.6 agreement rate $\rho(t)$ measures the gap. If $\rho(t)$ saturates well below 1, the gap is real and a deployed policy under-performs the training-time tree-augmented one. Mitigation could include test-time compute (sample multiple actions, run mini-tree, pick best) — but that abandons the single-shot inference assumption.

8.4 Reward-hacking surface

Each non-NHR term opens an exploit:

- r_{macro} : emit 2-step prefixes that match a macro start without the multi-step strategy actually being needed.
- r_{dead} : only effective if dead-end patterns generalise; otherwise inert.
- r_{pred} : predict trivially correct things (e.g., feasibility unchanged when no risky action is taken).
- r_{escape} : manufacture fake stagnation by intentionally taking near-no-ops.
- Tree expansion itself: gameable if the LLM can predict which states will be tree-expanded and exploit the depth-budget allocation.

Mitigations are case-by-case; full reward-hacking audit is a precondition for production use.

8.5 Cross-domain action grammar

The locality partition (local / mid / non-local), the macro vocabulary, the tree-expansion branching strategy, and the action-class regimes are grammar-specific. Truss $\rightarrow$ linkage / battery has a different grammar. A domain-agnostic action embedding (“change that reduces the dominant constraint violation, normalised by current slack magnitude”) would make the relevant terms portable. Open.

8.6 Per-problem threshold generalisation

Hard-coded $\text{FOS} \geq 1.5$ and $\delta \leq 0.01$ thresholds are baked into φ and Φ . Training should stratify by threshold rather than train on a single value, so Φ generalises across problems with tighter or looser specifications.

8.7 Branch-entropy as a lost signal

Branch entropy $H_{\text{gold}}(s)$ at gold tree branching states is genuinely informative — it tells the LLM how decision-difficult each state is. The data is too sparse (~ 100 – 200 branching states across all 20 trees) to support a regressor or robust lookup. If more tree data becomes available (more problems, deeper search per problem), this signal should be revisited as a calibration target for LLM action-class entropy.

8.8 SFT prior staleness

The action-class prior p_{gold} used in §2.1 reflects the search procedure that built the trees. If GRPO drifts the policy substantially, the gold prior may diverge from what is actually optimal at GRPO-time states. The SFT KL regularisation provides an anchor but at the cost of pulling toward gold behaviour. Open: how strong should the SFT KL coefficient be, and should it decay during GRPO?

9. Recommended reading

Reward shaping and policy invariance - Ng, Harada & Russell, *Policy invariance under reward transformations*, ICML 1999. - Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed., 2018), Ch. 17.

Online tree-augmented learning - Silver et al., *Mastering the game of Go with deep neural networks and tree search*, Nature 529, 2016. The amortised-search-into-priors paradigm. - Anthony, Tian & Barber, *Thinking Fast and Slow with Deep Learning and Tree Search*, NeurIPS 2017. Expert iteration; closest analog to the GRPO + tree-expansion architecture here. - Schrittwieser et al., *Mastering Atari, Go, chess and shogi by planning with a learned model*, Nature 588, 2020. MuZero — model-based MCTS with learned transitions; here the FEA is the perfect model.

Preference-based learning from gold trees - Rafailov et al., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, NeurIPS 2023. For §2.2.

Sequential pattern mining - Pei et al., *PrefixSpan: Mining Sequential Patterns Efficiently*, ICDE 2001. For §2.3.

Mutual information estimation - Kraskov, Stögbauer & Grassberger, *Estimating mutual information*, Phys. Rev. E 69, 2004. - Belghazi et al., *MINE: Mutual information neural estimation*, ICML 2018.

Engineering and architectural design behaviour - Schön, *The Reflective Practitioner*, Basic Books 1983. Forward prediction (§4.1). - Cross, *Designerly Ways of Knowing*, Springer 2007. Strategy templates (§2.1). - Goel & Pirolli, *The structure of design problem spaces*, Cognitive Science 16, 1992. Subtree value foresight. - Yilmaz, Daly, Seifert & Gonzalez, *How do designers generate new ideas? Design heuristics across two disciplines*, Design Science 1, 2015. Macro vocabulary (§2.3). - Atman et al., *A comparison of freshman and senior engineering design processes*, Design Studies 20, 1999. - Pahl & Beitz, *Engineering Design: A Systematic Approach* (3rd ed.), Springer 2007. Margin preservation (§1.4).

LLMs as priors for informed search - Wang et al., *Voyager: An open-ended embodied agent with large language models*, 2023. - Yao et al., *Tree of Thoughts: Deliberate problem solving with large language models*, NeurIPS 2023. - Sutton, *The Bitter Lesson*, 2019. The counter-argument worth confronting.